

When Is Tool Reranking Worth the Cost?

Selective cross-encoder inference on a tool-disjoint ToolRet holdout

Michael Baffour Awuah

Independent undergraduate research | October 3, 2026

Abstract

A cross-encoder can refine a retrieved tool list, but it adds inference time and can also demote a useful endpoint. This study asks whether inexpensive signals from lexical and dense retrieval can identify queries worth reranking. A ridge router is fitted on 300 development queries, calibrated on 300 separate queries, and tested on 1,500 queries with no shared positively labeled tool IDs across the three sets. The searchable catalog contains 37,292 tools. The preselected router makes 1,149 cross-encoder calls, 23.4% fewer than always-on reranking, with nDCG@10 of 0.5273 versus 0.5241. Both paired 95% intervals satisfy the predefined 0.01 loss tolerance. Mean warm CPU retrieval time falls by 14.0% on a separate 90-query, three-repetition benchmark. A simple top-result disagreement gate has higher observed nDCG and fewer calls than the learned router. The results support selective inference in this configuration, while leaving the value of learned allocation unresolved.

1 Introduction

Consider a request for both transaction history and the current month's usage quota. A retriever must return endpoints for both operations. A model that moves a plausible transaction endpoint to the top can still make the list worse if it pushes the quota endpoint out of the first ten results. This occurs in the held-out example examined in Section 6. Tool retrieval therefore depends on coverage of a request as well as similarity to its wording.

The usual retrieve-then-rerank pipeline pays for the second model on every query. That is reasonable when reranking helps consistently. When its benefit varies, a cheaper first-stage signal may be enough to choose between the original list and the reranked list. The question here is whether such a choice can save actual inference work while keeping a declared level of ranking quality.

The study combines a frozen learned router, simple gates, and random allocations at matched call budgets. It separates ranking evaluation from conditional execution and measured elapsed time, then examines why the aggregate result differs across data sources. This is an empirical extension of existing gating methods to tool retrieval; the contribution is the controlled comparison and its reproducible evidence.

2 Related work

ToolRet [1] provides heterogeneous tool-retrieval tasks and documents. ToolRerank [2] studies adaptive truncation and hierarchy-aware reranking, including the different needs of single-tool and multi-tool requests. Bacellar [3] investigates pre-reranker feature gating for multi-hop retrieval and reports harmful skips alongside aggregate performance. Lookahead-R [4] addresses budget-aware tool retrieval through execution-centric planning. The present study uses a smaller decision: rerank a fixed candidate prefix or return the fused first-stage list. The systems and evaluation scopes differ, so their reported scores are not used as directly comparable baselines.

3 Experimental design

3.1 Data and separation

The corpus is the pinned ToolRet web-tool subset. The query universe contains 3,099 deduplicated APIGen, ToolBench, and ToolACE queries; APIBank queries are excluded. Queries that share a positive tool label are joined into connected components before partitioning. Keeping each component intact prevents the router from learning from a labeled endpoint that appears again in calibration or confirmation.

Table 1. Study cohorts. Each active cohort is balanced across the three sources.

Cohort	Queries	Components	Use
Development	300	252	Fit and standardize the router
Calibration	300	249	Set score thresholds without labels
Confirmation	1,500	1,174	Evaluate the frozen policies

The previously observed pilot supplies development data. Its components contain another 629 queries, which are quarantined. Whole remaining components are assigned by a deterministic hash order and quota-fitting rule; 370 queries remain in reserve. The active sets share no query IDs, normalized query texts, positive tool IDs, or components. All tools remain searchable: the separation concerns supervised router exposure, not removal of held-out tools from the catalog. These components are not verified API families.

3.2 Retrieval and the routing decision

BM25 and normalized all-MiniLM-L6-v2 embeddings each retrieve 100 tools. Equal-weight reciprocal-rank fusion uses a constant of 60. The cross-encoder, ms-marco-MiniLM-L-6-v2, scores the first 20 fused candidates and reorders that prefix; the tail is retained. Both neural models are used off the shelf. Inference uses a maximum pair length of 256, batch size 64, four Torch threads, and one interop thread on CPU.

The router predicts the development-query change in nDCG@10 from reranking. Ridge regularization is fixed at 10; feature standardization uses development data only. Seven scalar features describe first-stage disagreement, overlap, rank coherence, query length, and the fused top-score margin (Appendix A). At inference, they require neither relevance labels nor cross-encoder scores. Tool IDs are used only for rank and equality comparisons.

$$\text{Predicted gain} = \text{ridge}(\text{first-stage features}); \text{rerank if predicted gain} > \text{frozen threshold}.$$

Prediction quantiles on calibration set nominal call targets of 25%, 50%, and 75%. Ties skip. The 75% target is the sole primary policy; the other targets and simple gates are exploratory. Targets are calibration settings rather than exact quotas on new queries. The protocol was public before fresh rankings were generated, and the fitted router and thresholds were frozen before confirmation quality was evaluated.

3.3 Metrics and the primary criterion

Positive relevance grades are binarized. The primary metric is nDCG@10, with source means weighted equally. The router must use fewer cross-encoder calls and place both paired 95% interval lower bounds for its nDCG difference from always-on reranking above -0.01. The margin is a chosen engineering tolerance. The two bootstrap analyses resample queries and positive-tool components within source, respectively, using 10,000 draws. Component resampling retains a query-macro estimand through score-sum/query-count ratios.

4 Ranking quality and inference work

Table 2. Confirmation results on 1,500 queries. Each cross-encoder call scores 20 pairs. Only learned 75% versus always is the primary comparison.

Policy	nDCG@10	MRR@10	Recall@10	Required CE calls	Calls saved
BM25	0.4689	0.4725	0.5840	0	100.0%
Base MiniLM	0.4602	0.4583	0.5911	0	100.0%
Hybrid RRF	0.5208	0.5196	0.6522	0	100.0%
Always rerank	0.5241	0.5301	0.6499	1,500	0.0%
Disagreement gate	0.5314	0.5388	0.6538	1,077	28.2%
Jaccard gate	0.5239	0.5276	0.6513	1,391	7.3%
Learned 25%	0.5269	0.5272	0.6549	409	72.7%
Learned 50%	0.5305	0.5339	0.6569	829	44.7%
Learned 75%	0.5273	0.5327	0.6520	1,149	23.4%

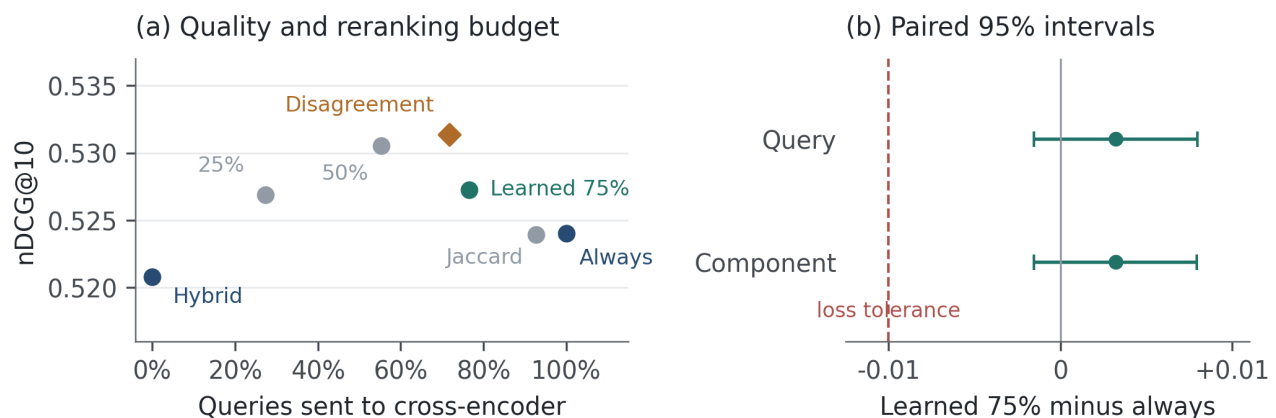


Figure 1. (a) Measured ranking quality against required call fraction. Learned-budget points other than the primary, and simple gates, are exploratory; the primary router is green. (b) Primary nDCG difference from always-on reranking. Both 95% intervals lie above the predefined loss tolerance and include zero.

4.1 The predefined tradeoff is achieved

The primary difference is +0.0032 nDCG@10: the query interval is [-0.0016, +0.0080] and the component interval is [-0.0016, +0.0079]. Both lower bounds exceed -0.01. Conditional execution over every confirmation query verifies 1,149 actual calls, 22,980 scored pairs, and 351 skipped calls. Every routing decision and final ranking matches frozen evaluation. The result meets the stated quality-and-work criterion; the intervals do not establish a quality improvement.

4.2 Learning is not the clear winner

The disagreement gate reranks when BM25 and dense retrieval choose different top results. Its nDCG@10 is 0.5314 with 1,077 calls, giving a better observed quality/cost point than the primary router. The learned-minus-disagreement component interval is [-0.0094, +0.0010]. The primary policy remains fixed after evaluation.

5 From avoided calls to elapsed time

Call counts do not measure user-visible delay. Skipping the cross-encoder still leaves BM25, dense encoding and search, fusion, and routing to run. A separate benchmark therefore recomputes the full query-to-ranking path for four policies, using 90 confirmation queries selected by hash, 30 per source. Each query is measured three times under each policy, for 1,080 requests in total.

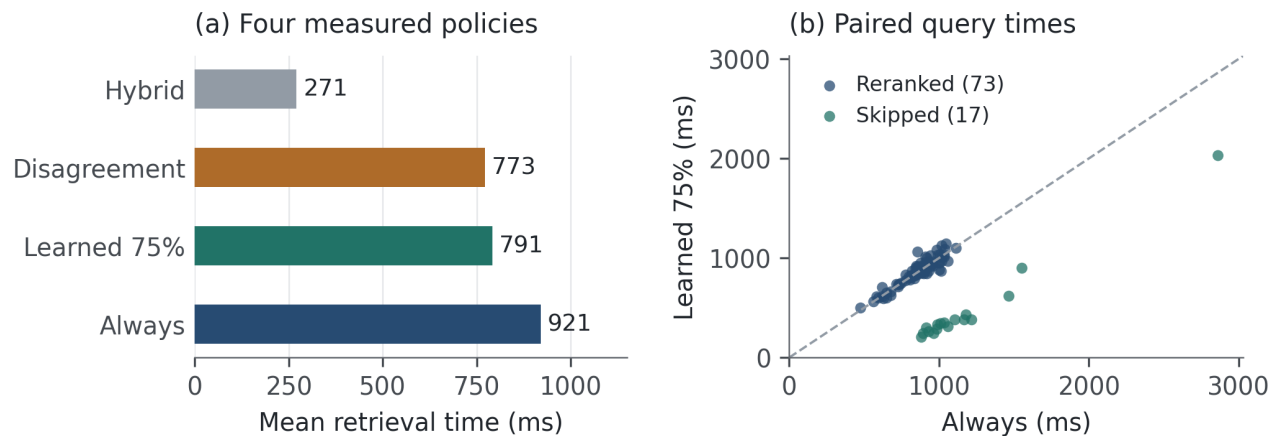


Figure 2. Warm sequential CPU retrieval on 90 queries, averaging three repetitions per query and policy. (a) Policy means. (b) Each point compares the same query under always and learned 75%. The dashed line marks equal time. Most savings occur on the skipped branch; reranked queries still incur cross-encoder inference.

Table 3. Measured elapsed time. Percentiles describe query means over three repetitions, rather than individual request tail latency.

Policy	Mean ms	p50 ms	p95 ms	CE calls / requests
Hybrid RRF	271.1	242.1	378.4	0 / 270
Always rerank	920.7	907.0	1172.3	270 / 270
Disagreement gate	772.5	847.7	1137.3	210 / 270
Learned 75%	791.4	850.9	1083.7	219 / 270

5.1 Measured speedup

Mean time decreases from 920.7 ms to 791.4 ms, a 14.0% reduction (paired 95% interval [8.2%, 20.0%]). This benchmark sends 219 of 270 primary-policy requests to the cross-encoder, saving 18.9% of calls. The 23.4% call saving in Section 4 describes the larger 1,500-query cohort. The different fractions reflect the benchmark subset and should not be treated as the same measurement.

5.2 Measurement boundaries

Models, the index, and corpus embeddings are already loaded. Timers include BM25, dense query encoding, exact corpus search and sorting, fusion, feature calculation, the routing decision, and conditional cross-encoder scoring and prefix sorting. Query and policy order are randomized and cyclically balanced. Every live ranking is checked against the frozen cache after timing.

The machine reports an AMD EPYC 9V74 CPU with nine logical CPUs available; Torch uses four threads and one interop thread. Loading, warmup, validation, networking, queueing, and concurrent serving are outside the measured interval. These results characterize warm sequential CPU retrieval on this machine. They leave production serving behavior and GPU performance open.

6 Where reranking helps and hurts

The small aggregate gain from always-on reranking conceals a source-level reversal. It improves mean nDCG on APIGen and ToolACE while reducing it on ToolBench. The primary router sends 472, 303, and 374 queries from those sources to the cross-encoder, respectively. Source-matched controls matter because changing the allocation across these sources can change the overall score without better decisions within a source.

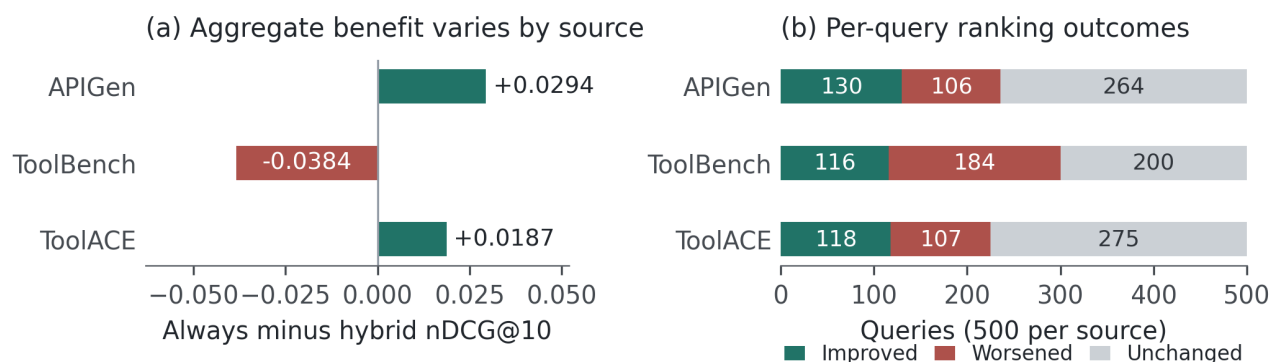


Figure 3. Exploratory always-versus-hybrid diagnostics. Each source contributes 500 queries. The aggregate change (a) and per-query outcomes (b) reveal different reranking behavior across sources. Outcomes use binary-label nDCG@10, with a numerical equality tolerance of $1e-12$.

6.1 Candidate coverage and request coverage

On ToolBench, 184 queries lose nDCG under reranking and 116 improve. For 132 queries, none of the positive labels is in the scored prefix. A reranker cannot repair that candidate miss. For other queries, positively labeled candidates are present but their ordering deteriorates. These are different problems: improving candidate retrieval does not by itself solve the ordering of multiple requested operations.

Table 4. Two outcome-selected ToolBench cases. Names are shortened for readability; full IDs, query text, and tokenization are in the case artifacts. A dash indicates absence from fused candidates.

Query / labeled endpoint	Hybrid rank	CE rank	Primary action
81 / transaction history	1	4	Skip reranking
81 / monthly usage quota	2	15	Skip reranking
716 / all Nexweave templates	15	1	Rerank
716 / template details	13	2	Rerank
716 / icon search	-	-	Rerank

In query 81, the cross-encoder promotes a transaction endpoint from another corpus source that has no positive label for the query. Meanwhile, the labeled quota tool falls from rank 2 to rank 15, reducing nDCG by 0.7359. Neither positive pair is truncated. Skipping preserves both labeled endpoints at the top. Query 716 shows the converse: two template tools move into the first two positions, raising nDCG by 0.7654, but the missing icon-search endpoint remains unrecovered.

Of the 184 ToolBench losses, 152 have no truncated positive pair, so clipping a labeled tool cannot explain every loss. The cases illustrate ordering changes, rather than establish their cause or prevalence. They were selected by outcome size with query-ID tie breaks.

7 Discussion

7.1 The useful result is a deployment tradeoff

For this frozen retrieval configuration, a gate can avoid real cross-encoder inference and meet a predefined ranking-loss tolerance. Conditional execution and elapsed-time measurement make the cost claim concrete. The result is narrower than saying reranking is unnecessary: some requests benefit substantially, and the router skips 60 queries whose nDCG would have improved. It also avoids 87 damaging reranks; the remaining 204 skipped queries are unchanged.

The simple disagreement rule is a consequential baseline, rather than a formality. It has the stronger observed quality/cost point and a lower measured mean time. The learned router's confidence intervals against disagreement and source-matched random allocation include zero. More model complexity is therefore not justified by this experiment alone. A new study should compare simple gating against learned allocation under another frozen cohort or retrieval configuration.

7.2 Limits and next experiments

The component split limits reuse of positive labels in supervised router fitting. It does not establish API-family separation or clean pretraining of the public neural models. Whole-component allocation and pilot quarantine also produce a particular benchmark cohort rather than a sample of live user traffic. Component bootstrap captures observed label sharing, while other semantic dependencies may remain.

Binary relevance judgments can omit interchangeable endpoints. Unjudged tools may be useful alternatives, but name similarity does not establish equivalence, and the original labels are retained. Recall and all-label coverage describe benchmark labels, and this study does not run the retrieved tools to completion. Controlled changes to document serialization and the sequence limit could help distinguish truncation from request-coverage effects. Independently judging candidate equivalence and testing downstream execution would address the larger question of whether a ranking improvement makes the agent more useful.

8 Conclusion

The frozen learned router reduces actual cross-encoder calls by 23.4% across 1,500 held-out queries and meets the declared 0.01 nDCG loss tolerance. A separate repeated CPU benchmark measures 14.0% lower mean retrieval time. The simpler disagreement gate has the better observed quality/cost point, and learned allocation superiority remains unresolved. The practical finding is that reranking should be evaluated as a query-dependent tradeoff, with simple controls and actual execution alongside aggregate ranking scores.

References

- [1] Shi et al. (2025). Retrieval Models Aren't Tool-Savvy: Benchmarking Tool Retrieval for Large Language Models. Findings of ACL 2025. <https://aclanthology.org/2025.findings-acl.1258/>
- [2] Zheng et al. (2024). ToolRerank: Adaptive and Hierarchy-Aware Reranking for Tool Retrieval. LREC-COLING 2024. <https://aclanthology.org/2024.lrec-main.1413/>
- [3] Bacellar (2026). Per-Query Gating of LLM Rerankers for Multi-Hop Retrieval. arXiv preprint 2609.22880. <https://arxiv.org/abs/2609.22880>
- [4] Wu, Guo and Li (2026). Lookahead-R: Budget-Aware Tool Retrieval via Execution-Centric Planning. ICMR 2026; arXiv 2609.35811. <https://arxiv.org/abs/2609.35811>

Appendix A Router details and allocation controls

The following definitions are frozen in the protocol. Ranks are one-based; top-k lists contain unique tool IDs. Feature standardization is fitted on the 300 development rows. The ridge intercept is unpenalized, and the deployment decision uses a strict greater-than comparison with the threshold.

Table A1. Seven inference-time features. Two Jaccard cutoffs yield two separate features.

Feature	Definition
Top-1 disagreement	1 if BM25 and dense top-1 IDs differ; otherwise 0.
Top-10 Jaccard	Intersection size divided by union size over the two top-10 lists.
Top-20 Jaccard	The same overlap measure at cutoff 20.
Weighted top-20 overlap	Sum of minimum reciprocal-rank weights divided by sum of maximum weights over the union; missing-list weight is 0.
Top-20 rank coherence	1 minus the mean absolute rank difference over the union, divided by 20. A missing rank is 21.
Log query length	$\log_{10}$ of the whitespace-delimited query token count.
Normalized RRF margin	(largest minus second-largest fused score) / largest score, using both top-100 lists. A singleton has margin 1.

Table A2. Calibration targets and realized confirmation work. Thresholds are set from label-free calibration predictions.

Call target	Frozen threshold	Test calls	Realized call fraction
25%	0.04356275	409	27.3%
50%	0.01093137	829	55.3%
75%	-0.01178293	1,149	76.6%

The cheap Jaccard gate reranks when top-10 BM25/dense Jaccard overlap is below 0.5, a fixed rule without tuning. All secondary budget, gate, and allocation comparisons are exploratory. Global random controls sample the exact call count for each policy; source-matched controls additionally preserve calls per source. The range below describes variability across 20 seeds, rather than a confidence interval.

Table A3. Random allocation controls, seeds 0-19.

Control	Calls	Mean nDCG	Seed range
Global / disagreement	1077	0.5225	[0.5173, 0.5278]
Global / learned 25%	409	0.5214	[0.5137, 0.5272]
Global / learned 50%	829	0.5216	[0.5165, 0.5276]
Global / learned 75%	1149	0.5225	[0.5182, 0.5283]
Source-matched / disagreement	1077	0.5206	[0.5126, 0.5277]
Source-matched / learned 75%	1149	0.5242	[0.5186, 0.5274]

A test-label-aware oracle at the primary call budget reaches nDCG@10 of 0.5918. It selects queries using their true reranking gains and is only a diagnostic ceiling. It is unavailable to a deployed router.

Appendix B Additional results and reproduction

Table B1. Exploratory paired nDCG comparisons. Intervals use 10,000 paired source-stratified bootstrap draws, seed 20261003; no multiple-testing correction is applied.

Difference	Mean	Query 95% CI	Component 95% CI
Disagreement - always	+0.0073	[+0.0023, +0.0124]	[+0.0021, +0.0125]
Hybrid - always	-0.0032	[-0.0155, +0.0090]	[-0.0158, +0.0091]
Learned 75% - disagreement	-0.0041	[-0.0092, +0.0011]	[-0.0094, +0.0010]
Learned 75% - global random	+0.0048	[+0.00003, +0.0096]	[+0.0002, +0.0093]
Learned 75% - source random	+0.0031	[-0.0015, +0.0078]	[-0.0014, +0.0076]
Disagreement - global random	+0.0088	[+0.0040, +0.0138]	[+0.0038, +0.0137]
Disagreement - source random	+0.0108	[+0.0058, +0.0160]	[+0.0055, +0.0160]

Table B2. Source-level confirmation scores. Each source has 500 queries; these comparisons are exploratory.

Source	Hybrid	Always	Disagreement	Learned 75%
APIGen	0.5851	0.6145	0.6225	0.6169
ToolBench	0.3588	0.3204	0.3333	0.3301
ToolACE	0.6186	0.6372	0.6383	0.6348

B.1 Reproduction and verification

An independent numerical implementation reconstructs components, router fitting and thresholds, policy metrics, random allocations, bootstrap intervals, and runtime evidence. All 317 checks pass. The default reproduction runner was also executed: numerical results match, and frozen predictions, decisions, per-query metrics, and evaluation Markdown are byte-identical. Five live pinned parquet downloads reconstruct the canonical corpus and cohort fingerprints.

```
.venv/bin/python scripts/run_confirmation_study.py \
  --output-dir data/research_reproduction_run1
```

Install the recorded environment using docs/confirmation_reproduction.md. The default runner freshly evaluates published caches. --recompute regenerates rankings and the fit; --measure-runtime executes conditional inference and timing. These flags were not run in the default smoke test; the original study's inference and runtime are separate completed evidence.

B.2 Evidence and pinned inputs

The repository includes per-query metrics and decisions, raw latency events, failure cases, freeze receipts, source fingerprints, and the independent audit. The report sidecar binds its PDF, Markdown, and figures to the same measured inputs. Full hashes and model/data revisions are kept in the machine-readable artifacts rather than reproduced in the main paper. Exact model revisions and package versions are recorded in the protocol and research requirements.

Protocol freeze: 3e813e1af650104f33aaf77febce5a8c1a3f1213

Router SHA-256: f24613a552d36737f4a17797b94f0c9348a67072a97cfc3cb471ae4bd25c159c

[Code, measured evidence, and reproduction instructions](#)